



Panchip Microelectronics Co., Ltd.

PAN3029/PAN3060 MAPM Application Reference

Current version: 1.3

Release date: 2024.04

Shanghai Panqi Microelectronics Co., Ltd.

Address: Room 302, Building D, No. 666, Shengxia Road, Zhangjiang Hi-Tech Park,
Shanghai Contact number: 021-50802371

Website: <http://www.panchip.com>

Documentation

Due to version upgrades or other reasons, the content of this document will be updated from time to time. Unless otherwise agreed, the content of this document is only for guidance, and all statements, information and suggestions in this document do not constitute any express or implied warranty.

Trademark

Panqi is a trademark of Panqi Microelectronics Co., Ltd. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

Disclaimer

All or part of the products, services or features described in this document may not be within the scope of your purchase or use. Unless otherwise agreed in the contract, Panqi Microelectronics Co., Ltd. does not make any express or implied statements or warranties regarding the contents of this document.

Revision History

Version	Revision Date	Description
V1.0	2023.06	Initial version
V1.1	2023.09	Update SDK routines and some interface functions
V1.2	2023.12	Update SF, BW changes, sleep time adaptation function
V1.3	2024.04	(1) Note: Explain SF5/SF6, do not recommend use (2) Modify interface functions names

Table of contents

1	Function Introduction	1
2	MAPM frame structure	1
3	SDK interface functions	3
3.1	rf_set_mapm_on	3
3.2	rf_set_mapm_off	3
3.3	void rf_set_mapm_para	4
3.4	rf_calculate_mapm_preambletime	5
3.5	rf_set_mapm_addr	5
4	Software	6
4.1	Verification content	6
4.2	Software Design Process	6
4.3	Verification Steps	7
5	System Demo Board	17
5.1	Module RX, TX	17
5.2	Button Functions	17
6	Notes	18
6.1	Address	18
6.2	The last Group in Field has its ADDR position function selected	18
6.3	Sync word configuration	18
6.4	Preamble length configuration	19
6.5	Sleep Time	19

1 Function Introduction

In the ChirpIOT star networking scenario, in order to ensure that the node-end device can receive the data sent by the gateway, the gateway will use an extra-long Preamble to send, such as Preamble taking 1s. In this case, all node-end devices will continue to receive after receiving the Preamble until the node network address is displayed in the Payload stage. At this time, the software of the node-end device can compare the received network address with the node device's own address. If they are different, it will stop receiving to reduce power consumption. If the address reception conditions are met, it will continue receiving. There are several problems with this solution:

- 1) Node devices that do not meet the address receiving conditions can only stop receiving until the payload stage, wasting a lot of power.
- 2) Node devices that meet the address receiving conditions also need to receive the long preamble before entering the payload receiving stage, which also wastes power.

To solve the above two power consumption problems, a new frame structure mode Multiple Address Preamble Mode (MAPM) is added to the chip PAN3029/PAN3060. The idea of this new frame structure module is to insert an address similar to the subsequent synchronization word (Sync Word) in the Preamble stage, so that the node end can judge in advance whether the address reception condition is met according to the address in the Preamble stage, which has solved the 1) problem. In addition, in order to solve the 2) problem, a variable counter is added to the address to indicate how much preamble length is left in the future. The node end device that meets the address reception condition can enter the low-power sleep state in advance, and then judge how long in advance to enter the receiving state according to the counter to ensure normal data reception, so as to further save the power consumption of the node end device that meets the address reception condition.

2 MAPM frame structure

The original frame structure is shown in Figure 1 below.

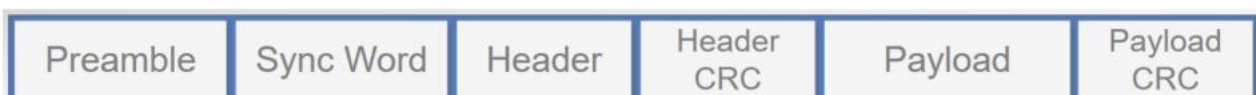


Figure 1 Original frame structure

As shown in Figure 2 below, the MAPM solution divides the preamble into F_n fields, where F_n is the number of fields. The structure of each field is the same, but the content can be different. At the same time, in order to ensure that the payload data can be received correctly, at least P_n preambles are guaranteed before the sync word, and P_n needs to be greater than or equal to 8.

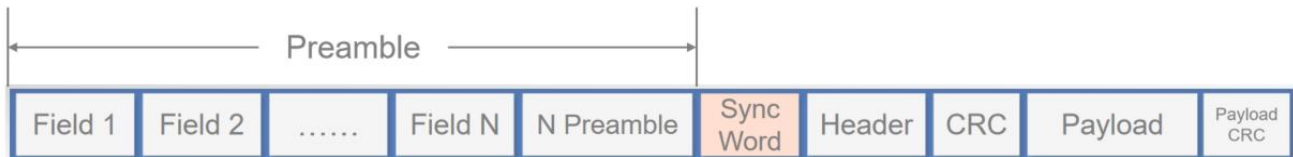


Figure 2 MAPM frame structure

As shown in Figure 3 below. Within a Field, it is divided into several Groups. The number of G_n in a Group is at least 1 and at most 4. G_n can be configured by registers. **Note: The G_n configurations of TX and RX are the same and predetermined.**

A Group is divided into P_g unmodulated chirps plus two modulated ADDRs. In the first Group, the minimum value of P_g is 8. The minimum value of P_g in other Groups is 0, which means that all ADDRs are connected together. Preambles can also be inserted in them. The maximum number is 255 (8 bits). ADDR is 1 byte of data and can be freely configured by registers.

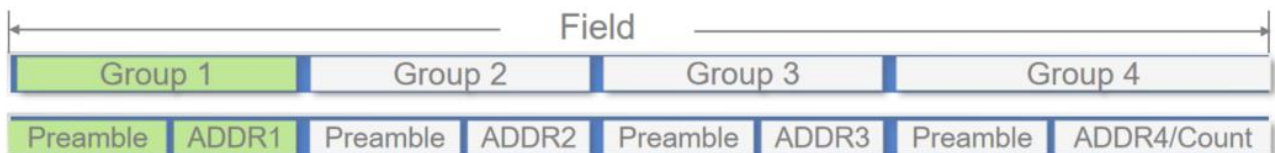


Figure 3 Field frame structure

The ADDR of Group 1 is fixed as an identifier, where ADDR1 is used for MAPM hardware synchronization, and ADDR2~ADDR4 is used for software applications. **Note: The address configuration of TX and RX must be agreed upon in advance, and ADDR1 must be the same for the MAPM function to take effect.**

The number of groups G_n :

when it is 0, there is only 1 group;

when it is 1, there are 2 groups;

when it is 2, there are 3 groups;

when it is 3, there are 4 groups.

No matter how many groups there are, the ADDR of the last group has two modes:

- 1) Address mode (ADDR), which can be configured with a 1-byte register;
- 2) Counter mode (COUNT), which counts down according to the current number of fields and is automatically filled in by RTL. This value identifies the number of remaining fields.

3 SDK interface functions

3.1 rf_set_mapm_on

Syntax

```
void rf_set_mapm_on(void)
{
    rf_mapm_en();
    rf_set_mapm_mask(MAPM_ON);
}
```

Purpose

Enable mapm mode and enable mapm interrupt.

3.2 rf_set_mapm_off

Syntax

```
void rf_set_mapm_off(void)
{
    rf_mapm_dis();
    rf_set_mapm_mask(MAPM_OFF);
}
```

Purpose

Disable mapm mode and enable mapm interrupt.

3.3 void rf_set_mapm_para

Syntax

```
void rf_set_mapm_para(stc_mapm_cfg_t *p_mapm_cfg)
{
    rf_set_mapm_field_num(p_mapm_cfg->fn);
    rf_set_mapm_field_num_mux(p_mapm_cfg->fnm);
    rf_set_mapm_group_fun_sel(p_mapm_cfg->gfs);
    rf_set_mapm_group_num(p_mapm_cfg->gn);
    rf_set_mapm_firgroup_preamble_num(p_mapm_cfg->pgl);
    rf_set_mapm_group_preamble_num(p_mapm_cfg->pgn);
    rf_set_mapm_neces_preamble_num(p_mapm_cfg->pn);
}
```

Purpose

Set mapm mode related parameters.

Parameters

fn {0x01~0xe0}: Number of fields in mapm preamble {field1, field2,.....fieldfn}

Fnm{0x00~0x03}: Number of repetitions of each field in mapm preamble

00 --> Repeat 1 time {field1, field2,.....fieldfn}

01 --> Repeat 2 times {field1, field1, field2, field2.....fieldfn, fieldfn}

10 --> Repeat 4 times

11 --> Repeat 8 times

gfs {0x00~0x01}: The last group in the field, its ADDR position function selection

0: Normal address {addr1, addr2, addr3, addr4}

1: Field number counter {addr1, addr2, addr3, fn}

gn {0x00~0x03}: Number of groups in a field

00--> 1 group

01--> 2 groups

02--> 3 groups

03--> 4 groups

pgl {0x08~0xff}: The number of preambles in the first group of each field, the default is 0x08

Pgn{0x00~0xff}: The number of preambles in other groups of each field, the default is 0x08

Pn{0x00~0xffff}: The number of chirps before the syncword after all fields are sent, the default is 0x08

3.4 rf_calculate_mapm_preambletime

Syntax

```
uint32_t rf_calculate_mapm_preambletime(void)
```

Purpose

Calculate the remaining time of the currently received preamble (ms)

Parameters

None

Return value

The remaining time of the currently received preamble

3.5 rf_set_mapm_addr

Syntax

```
uint32_t rf_set_mapm_addr(uint8_t addr_no, uint8_t addr)
```

Purpose

Configure the address in the mapm Group

Parameters

addr_no The address number to be configured

0: ADDR1

1: ADDR2

2: ADDR3

3: ADDR4

addr ADDR The value you want to configure

Return value

FAIL Operation failed

OK Operation successful

Note: For address configuration range, see 6.1 Address configuration

4 Software Design Verification

Note:

1. Only when the TX and RX mapm address parameters ADDR1 are the same, can RX enter the mapm interrupt, so the TX and RX codes must be configured with the same ADDR1 for the mapm function to take effect.
2. All verifications are based on the HC32F460EVB baseboard. For detailed operation methods, see Chapter 5 System Demonstration Board.

4.1 Verification content

This verification uses 3 modules, one for sending TX and two for receiving RX. The verification code shares the same project, and the TX\RX function is selected through the bottom plate DIP switch; the macro ADDR_MATCH in the code is used to select whether RX and TX ADDR2 are the same, so as to determine whether TX data is received. Two RXs, one with the macro turned on and the other with the macro turned off, are compared.

4.2 Software Design Process

1. Chip initialization, default SF7, BW500K when powered on.
2. Configure mapm initialization.
3. TX enters transmit mode, RX1, RX2 3029 enter low power mode, where RX1 ADDR2 is the same as TX, RX2 ADDR2 is different from TX;
4. RX 3029 wakes up and detects received data every half of the preamble duration under the current parameters.

4.3 Verification Steps

1. TX button sends data periodically.
2. RX1 and RX2 wake up periodically to detect received data every half of the preamble duration under the current parameters.
3. Use an ammeter to count the power consumption of the two RXs.
4. Use the serial port debugging tool to monitor RX received data.

4.3.1 SDK Examples

Reference code:

```
#define ADDR_MATCH 1 //Address matching
/*Initialize different SF mapm parameters*/
const stc_mapm_cfg_t mapm_cfg[] =
{
    {
        .mapm_addr= {0x73, 0x62, 0x54, 0x81},    //SF7 BW_500K
        .fn = 64,
        .fnm = 0,
        .gfs = FIELD_COUNTER,
        .gn = 2,
        .pg1 = 10,
        .pgn = 120,
        .pn = 8
    },
    {
        .mapm_addr= {0x73, 0x62, 0x54, 0x81},    //SF8 BW_500K
        .fn = 32,
        .fnm = 0,
        .gfs = FIELD_COUNTER,
```

```
.gn = 2,
.pg1 = 10,
.pgn = 120,
.pn = 8
},
{
.mapm_addr= {0x73, 0x62, 0x54, 0x81}, //SF9 BW_500K
.fn = 16,
.fnm = 0,
.gfs = FIELD_COUNTER,
.gn = 2,
.pg1 = 10,
.pgn = 120,
.pn = 8
},
{
.mapm_addr= {0x73, 0x62, 0x54, 0x81}, //SF10 BW_500K,Only for PAN3029
.fn = 8,
.fnm = 0,
.gfs = FIELD_COUNTER,
.gn = 2,
.pg1 = 10,
.pgn = 120,
.pn = 8
},
{
.mapm_addr= {0x73, 0x62, 0x54, 0x81}, //SF11 BW_500K,Only for PAN3029
.fn = 6,
.fnm = 0,
```

```
.gfs = FIELD_COUNTER,
.gn = 2,
.pg1 = 10,
.pgn = 120,
.pn = 8
},
{
.mapm_addr= {0x73, 0x62, 0x54, 0x81}, //SF12 BW_500K,Only for PAN3029
.fn = 6,
.fnm = 0,
.gfs = FIELD_COUNTER,
.gn = 2,
.pg1 = 10,
.pgn = 120,
.pn = 8
},
};/*3029/3060 initialization configuration*/
ret = rf_init(); //3029/3060 initialization
if(ret != OK)
{
    printf(" RF Init Fail");
    while(1);
}
rf_set_default_para(); //Configure default RF parameters
//Use the RF parameters initialized by the current demo now_bw = TEST_BW;
now_bw = TEST_BW;
now_sf = TEST_SF;
rf_set_bw(now_bw);
rf_set_sf(now_sf);
```

```

rf_set_code_rate(TEST_CR);

rf_set_freq(TEST_FREQ);

memcpy((uint8_t*)&cur_mapm_cfg,(uint8_t*)&mapm_cfg[now_sf-SF_7],
sizeof(stc_mapm_cfg_t)); //According to SF, get the current mapm parameters. The first mapm parameter is SF7

rf_set_mapm_on(); //Enable mapm mode

rf_set_mapm_para(&cur_mapm_cfg); //Configure mapm parameters

for(uint8_t i=0; i<cur_mapm_cfg.gn+1; i++)
{
    rf_set_mapm_addr(i, cur_mapm_cfg.mapm_addr[i]);
}

one_chirp_time = get_chirp_time(now_bw,now_sf); //Calculate packet duration in us
preamble_time=rf_calculate_mapm_preambletime(&cur_mapm_cfg, one_chirp_time);
//Calculate the preamble duration

rx_timeout_val = preamble_time + calculate_payload_time(tx_len) + 200;
//Calculate the complete packet time, and add 200ms to it as the reception timeout judgment time

sleep_timer_val = preamble_time/2;

if(app_tx_mode)
{
    DDL_Printf("tx\r\n");
}
else
{
    DDL_Printf("rx\r\n");
}

#ifdef ADDR_MATCH

//Modify ADDR1, the software is used to distinguish whether to receive TX send data
cur_mapm_cfg.mapm_addr[1] = 0x61;
rf_set_mapm_addr(1, cur_mapm_cfg.mapm_addr[1]);

#endif

rf_set_cad(CAD_DETECT_THRESHOLD_10, CAD_DETECT_NUMBER_3);

```

```
    rf_set_dcde_mode(DCDC_ON);

    rf_enter_single_timeout_rx(rx_timeout_val);

}

while(1){
    rf_irq_process();

    key_scan();

    key_event_process();

    process_rf_events();

}

/*MAPM sleeps after receiving data*/

flag = rf_get_rcv_flag();

if(flag == RADIO_FLAG_MAPM)
{
    rf_set_rcv_flag(RADIO_FLAG_IDLE);

    if(RxDoneParams.mpam_rcv_index == cur_mapm_cfg.gn + 1)
//Receive a complete field
    {
        uint8_t i;

        if(cur_mapm_cfg.gfs == FIELD_COUNTER)
        {
            addr_num = cur_mapm_cfg.gn;
        }
        else
        {
            addr_num = cur_mapm_cfg.gn + 1;
        }

        for(i=0; i<addr_num; i++)
        {
```

```

        if(cur_mapm_cfg.mapm_addr[i] != RxDoneParams.mpam_rcv_buf[i])
//addr does not match, no need to receive

        {

            mapm_target = 0;

            break;

        }

    }

    if(i == addr_num)

    {

        mapm_target = 1; //address match, data needs to be received

    }

    if(cur_mapm_cfg.gfs == FIELD_COUNTER)//Counter Mode

    {

        cur_mapm_cfg.fn = RxDoneParams.mpam_rcv_buf[cur_mapm_cfg.gn];

        mapm_preamble_time=
rf_calculate_mapm_preambletime(&cur_mapm_cfg, one_chirp_time);

        one_field_chirp = cur_mapm_cfg.pg1+2 + (cur_mapm_cfg.pgn+2) *
cur_mapm_cfg.gn;

        if(mapm_target)

        {

            mapm_sleeptime = mapm_preamble_time -
one_field_chirp*one_chirp_time/1000; //Calculate the sleep time

            if(mapm_sleeptime > CODE_RUNTIME)

            {

                rf_sleep();

                mapm_sleep_flag = 1;

                mapm_sleep_begtime = SysTick_GetTick();

            }

            else //Sleep time is too short and does not sleep

            {

```

```

        DDL_Printf("sleep time too short\r\n");
    }
}
else
{
    memset(RxDoneParams.mpam_rcv_buf, 0,
RxDoneParams.mpam_rcv_index);

    RxDoneParams.mpam_rcv_index = 0;

    mapm_sleeptime = mapm_preamble_time +
PAYLOAD_LEN*one_chirp_time/1000;

    rf_sleep();

    mapm_sleep_flag = 1;

    mapm_sleep_begtime = SysTick_GetTick();

}

}

}

}

/*mapm sleep wakeup processing*/
if(mapm_sleep_flag == 1)
{
    if((SysTick_GetTick()-mapm_sleep_begtime) >
(mapm_sleeptime-CODE_RUNTIME)) //Wake up after reaching the sleep time
    {
        rf_sleep_wakeup();

        mapm_sleep_flag = 0;

        if(mapm_target)
        {
            mapm_target = 0;
        }

    }
else

```

```

    {
        recv_flag = 0;
    }
    if(rf_enter_continuous_rx() != OK)
    {
        DDL_Printf("set rx FAIL\r\n");
    }
}
}

```

The sample code configures mapm initialization. After TX sends data, RX receives data and determines whether it needs to receive data, and then performs sleep wake-up processing.

The sending module sends a data packet (the default parameters of SF7 BW500K data packet preamble duration is about 4200ms, and the payload duration is about 40ms). Use an ammeter to capture the current waveforms of the two RX receiving modules and observe the results.

4.3.2 Verification Results

The current meter captures the RX1 result as shown in Figure 2-1



Figure 2-1 RX1 current meter power consumption capture result

According to the results, when the sending module sends a data packet, RX1 detects the TX data after waking up, determines that the address matches, calculates the remaining preamble duration, sleeps for this time, and then wakes up to receive the payload data.

The serial port prints the RX1 result as shown in Figure 2-2:

```
[10:24:39.014]收←◆rx
[10:27:23.067]收←◆RF RX:1
[10:27:23.085]收←◆mapm addr[0] =73
mapm addr[1] =62
mapm addr[2] =3e
mapm addr[
[10:27:23.090]收←◆3] =73
mapm addr[4] =62
mapm addr[5] =01
mapm addr num =6
Rx : SNR: 9.369039 , RSSI: -11
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0a 0x0b 0x0c 0x0d 0x0e 0x0f 0x10 0x11 0x12 0x1
[10:27:23.107]收←◆3 0x14 0x15 0x16 0x17 0x18 0x19 0x1a 0x1b 0x1c 0x1d 0x1e 0x1f 0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2a 0x2b 0x2c
0x2d 0x2e 0x2f 0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39
[10:27:23.124]收←◆0x3a 0x3b 0x3c 0x3d 0x3e 0x3f 0x40 0x41 0x42 0x43 0x44 0x45 0x46 0x47 0x48 0x49 0x4a 0x4b 0x4c 0x4d 0x4e 0x4f 0x50 0x51 0x52
0x53 0x54 0x55 0x56 0x57 0x58 0x59
[10:27:23.138]收←◆0x5a 0x5b 0x5c 0x5d 0x5e 0x5f 0x60 0x61 0x62 0x63
```

Figure 2-2 RX1 serial port capture results

According to the results, only 6 mapm addrs in the preamble were received, while TX actually sent 96 mapm addrs.

During the remaining 90 mapm addrs, RX1 entered low-power sleep mode, and then woke up before the payload to receive a 100-byte payload.

The current meter capture results of RX2 are shown in Figure 2-3:



Figure 2-3 The current meter capture results

According to the results, when the sending module sends a data packet, RX2 detects the TX data after waking up, determines that the address does not match, calculates the remaining preamble and payload duration, sleeps for this time, and then wakes up to perform receiving detection again.

The result of printing RX2 through the serial port is shown in Figure 2-4:



Figure 2-4 RX2 serial port capture results

According to the results, RX2 directly sleeps after determining that the address does not match and no longer receives data, so no received number information is printed.

5 System Demo Board

The functional description of the demonstration system board based on the HC32F460EVb baseboard is shown in Figure 5-1.

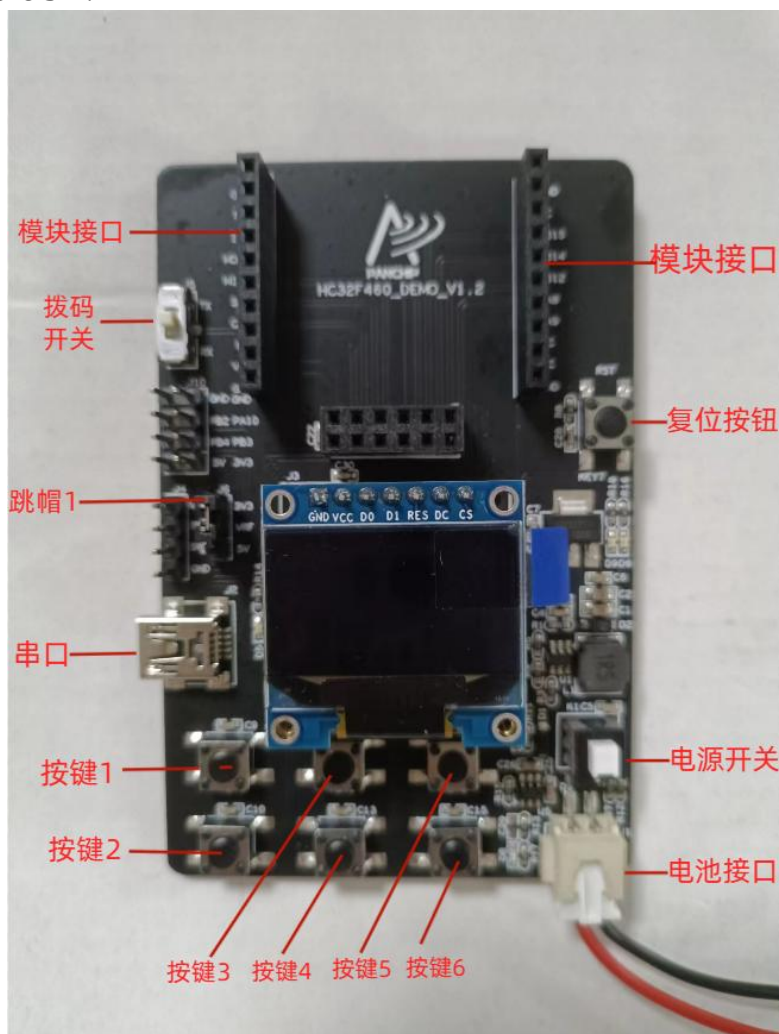


Figure 5-1 HC32 base board

5.1 Module RX, TX settings

DIP switch: Switch between transmit and receive modes. DIP switch up to TX for TX mode, and down to RX for RX mode.

5.2 Button Functions

Button 3: TX Press to send data.

Button 5: Switch SF, range 5~12, step 1.

Button 6: Switch BW, 62.5K/125K/250K/500K.

6 Notes

6.1 Address Configuration

1. Due to the characteristics of the modulation Chirp, all ADDR configuration ranges from 0x11 to 0xFF, and does not support values less than 0x11 and 0xX0. If the software receives these values, the software will reset the chip. At the same time, it should be noted that ADDR1 in Group1, ADDR2~4 in Group, and SyncWord in the normal frame structure are all different.
2. SF7~SF12, ADDR supports 8bit configuration. SF5 only supports 0b00xx00xx (0x11, 0x12, 0x13, 0x21, 0x22, 0x23, 0x31, 0x32, 0x33), and SF6 supports 0b0xxx0xxx. Here, it is still required to meet the requirements of not supporting values less than 0x11 and not supporting 0xX0. **(Note: Under SF5\SF6 configuration, when the counter mode count value of MAPM is less than 0x11, the counter value may be abnormal. It is not recommended to use the MAPM function under SF5\SF6 configuration).**
3. Address configuration is used with the parameter gn. According to the number of gn, the corresponding number of addr is configured continuously from addr1.
4. Each ADDR is an unsigned 8-bit data. When configuring, the upper 4 bits and the lower 4 bits should not be the same as far as possible.

6.2 The last Group in Field has its ADDR position function selected

- (1) . Parameter gfs = 0x00, the last group in field is configured as a normal Addr function.
- (2) . Parameter gfs = 0x01, the last group in field is configured as a counter. Pay attention to the configuration of fn and addr to avoid the situation where fn is continuously decremented and the counter is equal to ADDR1 or the synchronization word syncword, which may trigger a mapm interrupt or missynchronization.

6.3 Sync word configuration

SF7~SF12, the synchronization word syncword supports 8-bit configuration.

6.4 Preamble length configuration

- (1). The minimum number of preambles (pg1) in the first group of each field is 8.
- (2). The minimum number of preambles (pn) before the syncword after all fields are sent is 8.

6.5 Sleep Time

Calculate the subsequent preamble reception time based on the parameters. When entering sleep mode, pay attention to subtracting the sleep time, sleep wake-up time and other code running time. Try to wake up slightly earlier to avoid missing the syncword and failing to receive payload data.